

CSE 144 Final Project Report

Jay Jani (CruzID: jdjani)
Applied Machine Learning - Final Project, UC Santa Cruz

Spring 2026

1 Introduction

The task is to classify images into 100 fine-grained categories (cars, flowers, dishes, and aircraft) with only about ten training images per class, which is far too few to train a network from scratch. Transfer learning is the natural fit: a model pretrained on a large corpus already encodes general visual features, and I only need to adapt them to the 100 labels.

The usual recipe is to fine-tune the pretrained weights, but with ~ 10 images per class fine-tuning overfits. I take the opposite approach: I *freeze* the backbone and learn only a small classifier on its fixed embeddings. The final system combines linear probes on two frozen SigLIP-2 backbones, zero-shot text classifiers from the class names, and a cross-validated weighted ensemble. It reaches 96.3% five-fold accuracy and 98.1% on the Kaggle public leaderboard and the top public-leaderboard score.

2 Dataset

Size and structure. The data has 100 classes from four domains: 1,079 training images as `train/<label>/*.jpg` and 1,036 unlabeled test images in `test/`. Folder names are the integer labels, kept exactly to match the grader. The classes are contiguous blocks of 25 (0–24 food, 25–49 flowers, 50–74 cars, 75–99 aircraft), which I use to build domain-aware text prompts.

Class names. The competition provides only integer labels, with no human-readable class names, but the zero-shot text heads need the actual category names. I therefore built `class_names.csv` by hand: for each class I inspected its training images and used reverse image search (mainly Google Lens) to identify the specific fine-grained label, e.g. a particular car model or flower species. This was the most time-consuming part of data preparation, especially for the closely related car and aircraft variants.

Preprocessing. Each image is resized to the backbone’s native resolution (384 or 512) and normalized with that model’s mean and standard deviation. At inference I use 4-view test-time augmentation (center crop, square resize, and their horizontal flips) and average the embeddings. There is no random training-time augmentation, since the backbone is frozen and the probe sees fixed feature vectors.

3 Implementation

Frozen backbones. I use two pretrained SigLIP-2 vision–language models from `open_clip`: ViT-g/16 at 384px (1536-d) and SO400M/16 at 512px (1152-d), both trained with a contrastive image–text objective.

Contrastive models fit this task because the classes are nameable, web-common concepts. The backbones are never updated; I only run them forward and cache the embeddings.

Linear probes. For each backbone I fit a multinomial logistic-regression probe on the L2-normalized embeddings (`scikit-learn`, $C = 10$, balanced classes). A third probe is fit on the concatenation of both embeddings, capturing complementary information the two models encode differently.

Probe training. The probe minimizes a class-balanced cross-entropy loss, solved by the L-BFGS optimizer for up to 2000 iterations; the $C = 10$ inverse- ℓ_2 penalty above is the equivalent of weight decay. Because L-BFGS is a full-batch second-order method, there is no minibatch size, learning-rate schedule, or epoch count to set. The convergence curves in Figure 1 instead use an equivalent linear head trained by gradient descent (Adam, learning rate 0.02, weight decay 0, no scheduler, full batch, 60 epochs) to visualize learning dynamics.

Zero-shot text heads. Each backbone has a text encoder. For every class I encode its name with a few domain-specific prompts (e.g. “*a photo of the car model { }*”) and average the text vectors; an image is scored by cosine similarity. I calibrate each head’s softmax temperature on the training folds, which sharpens probabilities and improves accuracy for free.

Ensemble. This gives five heads (two probes, two text heads, one concatenated probe). I combine their probabilities with non-negative weights tuned to maximize five-fold out-of-fold (OOF) accuracy via random-restart coordinate ascent. Weak heads can receive weight zero, so the search also does model selection.

4 Experiments

All choices are made on a leak-free 5-fold stratified cross-validation; every test prediction comes from a probe trained only on training data.

Which backbone? Table 1 compares single linear probes on different frozen backbones. The two SigLIP-2 models are clearly strongest, and when I added any of the others to the ensemble the weight search drove it to zero, i.e. it added nothing beyond the two SigLIP-2 models. I hypothesize that self-supervised features (DINOv2) lag the contrastive models, as expected for nameable classes.

Frozen vs. fine-tuning. I also fully fine-tuned a backbone with layer-wise LR decay and label smoothing. It still overfit on the small set, peaking near 95.4%, below the frozen ensemble and far more compute intensive. With ~ 10 images per class, adapting a linear head on fixed features simply generalizes better than moving the weights.

Probe hyperparameters. Accuracy is flat across $C \in [3, 100]$; ℓ_2 with 1bfgs beats ℓ_1 , and balanced weights match the default at the plateau, so I fix $C = 10$. A nonlinear (MLP/RBF Kernel) probe did not help, since the features are already nearly linearly separable.

Table 1: Single linear-probe accuracy (5-fold OOF) per frozen backbone. Only the two SigLIP-2 models receive non-zero ensemble weight.

Backbone	Pretraining	Probe OOF
SigLIP-2 ViT-g/16 (384)	contrastive (WebLI)	0.948
SigLIP-2 SO400M/16 (512)	contrastive (WebLI)	0.943
DFN ViT-H/14 (378)	contrastive	0.932
ConvNeXt-XXL	contrastive (LAION-2B)	0.913
EVA02-L/14 (336)	contrastive	0.906
MetaCLIP-2 ViT-H/14	contrastive	0.894
DINOv2 ViT-g/14	self-supervised	0.846

5 Results

The ensemble reaches 96.3% OOF accuracy (Table 2), about a point above the best single head, with a small per-fold standard deviation (± 0.009). Figure 1 shows the linear-probe training curves: it converges quickly and the small train/validation gap confirms it does not overfit.

Table 2: Final ensemble members and combined accuracy (5-fold OOF).

Head	OOF accuracy
SigLIP-2 ViT-g probe	0.948
SigLIP-2 ViT-g text	0.952
SigLIP-2 SO400M probe	0.943
SigLIP-2 SO400M text	0.943
Concatenated probe	0.947
Weighted ensemble	0.963

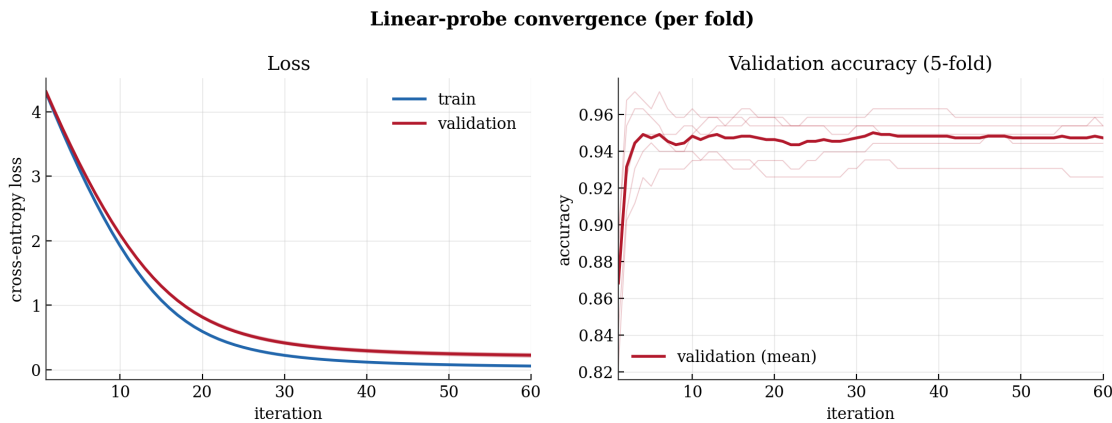


Figure 1: Five-fold loss and accuracy curves for a linear probe on frozen SigLIP-2 ViT-g embeddings. Faint lines are folds; bold lines are the mean. Final validation accuracy 0.947 ± 0.012 .

The final submission ranks **1st on the public leaderboard at 98.1% accuracy** (at time of writing). That split is only $\sim 11\%$ of the test set (~ 114 images), so a single image is worth nearly a percent. I treat the 96.3% cross-validation number as the more reliable estimate of true performance.

6 Discussion

What worked. The main win was choosing frozen feature extraction over fine-tuning: strong contrastive backbones separate these classes almost linearly, so a tiny head on fixed features generalizes better and is trivially reproducible. Temperature calibration, domain-aware prompts, and the early-fusion probe each added small gains.

Decorrelation limit. Once both SigLIP-2 models are in, adding more or stronger backbones barely moves the score, because ensembling only helps when members make unique mistakes and the remaining errors are largely shared. This is why every extra backbone was weighted to zero.

Failure cases. The clearest errors are two pairs of car and aircraft classes that are near-identical variants, visually indistinguishable from pixels alone. Such cases are effectively irreducible and account for part of the remaining $\sim 3.7\%$.

Limitations. The text heads rely on class names matching web concepts and would help less on abstract categories. Self-training (pseudo-labeling confident test images) flipped sign across seeds, i.e. was within noise, so I did not rely on it. A reasonable next step is light, regularized fine-tuning of only the last few blocks.

7 Reproducibility

A single seed (42) seeds Python, NumPy, and PyTorch with deterministic cuDNN; the probe and cross-validation are CPU-based and seeded, so results are reproducible on a given machine, and I report cross-validated accuracy with its standard deviation.

Environment. PyTorch 2.11 (CUDA 12.8), `open_clip`, `scikit-learn` 1.6.1, NumPy 2.0.2.

Hardware. A hardware preset auto-selects batch size and precision for the detected GPU; a cold run takes ~ 9 min on an A100 80 GB and ~ 25 min on an L4 24 GB, and seconds on a CPU once embeddings are cached.

How to Run. The full pipeline (encode, probe, ensemble, write `submission.csv`) runs with:

```
bash run.sh
```

or directly:

```
python src/ensemble_probes.py \  
--backbone ViT-gopt-16-SigLIP2-384:webli:cache/gopt \  
--backbone ViT-S0400M-16-SigLIP2-512:webli:cache/so400m512 \  
--write submission.csv
```

The figure comes from:

```
python src/probe_curve.py --emb cache/gopt/train.npy --out probe_curve.png
```

References

- [1] M. Tschannen et al. *SigLIP 2: Multilingual Vision–Language Encoders*. arXiv, 2025.
- [2] A. Radford et al. *Learning Transferable Visual Models from Natural Language Supervision (CLIP)*. ICML, 2021.
- [3] G. Ilharco et al. *OpenCLIP*. https://github.com/mlfoundations/open_clip, 2021.
- [4] F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. JMLR, 2011.